

Imię i nazwisko Marta Lipień Justyna Sielska Aleksandra Woźniak	Kierunek Inżynieria Obliczeniowa	Rok studiów i grupa I rok, Grupa 2
Data zajęć 15.01.2026 r.	Numer i temat sprawozdania 12. Projekt – Indiana Snake	

1.Cel i opis projektu

Cel projektu: Głównym założeniem było stworzenie **Indiana Snake** - autorskiej wersji klasycznego "Węża" w przygodowym klimacie. Chcieliśmy sprawdzić nasze umiejętności w C++ i zobaczyć, jak od podstaw zbudować silnik gry przy użyciu biblioteki **SDL2**. Skupiliśmy się na tym, aby gra działała płynnie (dzięki synchronizacji klatek), poprawnie odczytywała ruchy gracza i sprawnie zarządzała tekstami wyświetlanymi na ekranie.

Opis gry i mechaniki: W naszej grze nie sterujesz zwykłym wężem, ale poszukiwaczem przygód, który przemierza zapomniane ruiny oraz krypty.

- **Zbieranie skarbów:** Zamiast standardowych punktów, gracz poluje na **artefakty**. Każdy zebrany przedmiot dodaje 1 punkt do wyniku, ale sprawia też, że Indiana Snake wydłuża się o 2 segmenty. Przez to z każdym ruchem robi się coraz ciaśniej i trudniej;
- **Zasady przetrwania:** Gra jest bezlitosna - kończy się w momencie uderzenia w ścianę lub gdy wąż zapłącze się we własny ogon. Wymaga to od gracza planowania trasy z wyprzedzeniem;
- **Sterowanie:** Dodaliśmy blokadę skrętu o 180 stopni. Teraz program pilnuje, żebyś przez przypadek nie zawrócił w miejscu i nie wjechał prosto we własny ogon;
- **Klimat przygody:** Całość utrzymaliśmy w stylistyce retro. Po każdej porażce pojawia się ekran „**GAME OVER**” z Twoim ostatecznym wynikiem, co motywuje do kolejnego podejścia;
- **Licznik na żywo:** Dorzuciliśmy licznik punktów, który widać w rogu ekranu przez całą grę. Dzięki temu na bieżąco widzisz, jak Ci idzie i nie musisz czekać do końca partii, żeby sprawdzić wynik;
- **Interaktywne Menu:** Zbudowaliśmy klasyczne menu główne, w którym możesz wybrać tryb gry, sprawdzić tabelę wyników lub po prostu wyjść z aplikacji. Nawigacja jest intuicyjna - używasz strzałek do wyboru opcji, a wybrany element podświetla się na biało, żebyś zawsze wiedział, co klikasz;
- **System Leaderboard (Tabela Rekordów):** Gra ma własną pamięć. Twoje najlepsze wyniki nie przepadają - po każdej przegranej możesz wpisać swój nick, a program

automatycznie porówna Twój wynik z innymi. Tabela wyświetla TOP 10 najlepszych poszukiwaczy przygód, co pozwala na prawdziwą rywalizację ze znajomymi. Wszystko zapisuje się w pliku tekstowym, więc rekordy zostają z Tobą nawet po wyłączeniu komputera;

Zastosowane techniki programistyczne

- **Logika stanów** (enum GameState): Nasz program ma „ogarnięte” stany gry. Dzięki temu wie, kiedy ma być włączone menu, kiedy trwa rozgrywka, a kiedy czas na tabelę wyników. Wszystko jest na swoim miejscu i nic się nie nakłada.
- **Ruch węża** (deque): Do ciała Indiana Snake użyliśmy kolejki. Działa to prosto: dodajemy segment tam, gdzie głowa i ucinamy kawałek ogona na końcu. Dzięki temu wąż płynie bez zacięć.
- **Sprzątanie pamięci** (clean()): Nie zostawiamy śmieci. Nasza funkcja clean() na bieżąco usuwa stare napisy (np. nieaktualny wynik), żeby gra nie zapychała RAM-u i nie zwalniała po paru minutach.
- **Kolizje** (SDL_HasIntersection): Użyliśmy gotowego systemu do sprawdzania czy obiekty na siebie nachodzą. Program wyłapuje, kiedy wąż zje artefakt.
- **Wpisywanie nicku** (SDL_StartTextInput): Gra nie tylko czyta klawisze ruchu, ale też rozumie litery. Można normalnie wpisać swoje imię do rankingu.
- **Zapis rekordów** (fstream): Wyniki nie znikają po wyłączeniu komputera. Są zapisywane do pliku scores.txt, więc można bić rekordy znajomych następnego dnia.
- **Ranking** (sort): Żeby nie robić bałaganu, program sam sortuje graczy. Najlepsi zawsze lądują na samej górze listy TOP 10.
- **Stabilny FPS** (SDL_Delay): Dodaliśmy małe opóźnienie w pętli gry. Dzięki temu wąż nie pędzi i na każdym komputerze porusza się tak samo.

Wykorzystane biblioteki:

Biblioteki multimedialne (SDL2):

- **SDL2 (SDL.h)**: Główny silnik gry. Odpowiada za stworzenie okna, wyświetlanie grafiki oraz odbieranie sygnałów z klawiatury.
- **SDL2_image**: Pozwala na ładowanie obrazków w formatach innych niż BMP (np. .png dla naszych artefaktów i tła).
- **SDL2_ttf**: Odpowiada za renderowanie czcionek, dzięki czemu możemy wyświetlać napisy takie jak wynik czy "GAME OVER".
- **SDL2_mixer**: Służy do obsługi dźwięków i muzyki w tle, pozwalając na jednoczesne odtwarzanie wielu ścieżek dźwiękowych.

Standardowe biblioteki C++ (STL):

- **vector**: Kontener przechowujący listę skarbów (artefaktów) rozmieszczonych na mapie.
- **deque**: Kolejka o dwóch końcach, której użyliśmy do stworzenia ciała węża – pozwala ona na szybkie dodawanie nowej głowy i sprawne usuwanie ogona przy każdym ruchu.
- **algorithm**: Wykorzystywana do sortowania tabeli wyników (std::sort) oraz sprawdzania kolizji dla każdego segmentu ogona (std::for_each).
- **ctime**: Służy do pobierania czasu systemowego, co wykorzystujemy przy srand, aby skarby za każdym razem pojawiały się w innych, losowych miejscach.

Dodatkowe definicje:

- **SDL_MAIN_HANDLED**: Specjalna instrukcja, która pozwala programowi na użycie standardowej funkcji int main() zamiast wymuszania specyficznego formatu SDL, co ułatwia kompilację projektu.

2. WYBRANE FUNKCJONALNOŚCI:

- **Zarządzanie stanami gry**

Gra opiera się na strukturze: enum GameState, która pozwala na płynne przełączanie się między menu, rozgrywką, tabelą wyników i ekranem przegranej. Dzięki temu program wie, jakie zdarzenia obsługiwać w danym momencie.

```
enum GameState //stany gry
{
    MENU,
    LEADERBOARD,
    PLAYING,
    GAMEOVER,
    SAVESCORE
};
```

- **Dynamiczna struktura węża**

Do reprezentacji ciała Indiana Snake użyliśmy kolejki o dwóch końcach (std::deque). Jest to optymalne rozwiązanie, ponieważ w każdym kroku gry dodajemy nową pozycję głowy na początek kolejki i usuwamy ostatni element (ogon).

```
//dodanie najnowszej głowy na front
rq.push_front(head);

while (rq.size() > (size_t)snake_size)
{
    rq.pop_back();
}
```

- System zapisu i sortowania wyników (Leaderboard)

Gra prosi gracza o podanie nicku po przegranej, a następnie zapisuje wynik do pliku tekstowego. Wykorzystujemy std::sort z funkcją, aby tabela wyników zawsze pokazywała najlepszych graczy na górze.

```
//OBSŁUGA WPISU DO LEADERBOARDA
string usernameinput = "";
bool typing = false;

GameState current_state = MENU;

artefakty(artifacts);

//OBSŁUGA MENU INTERAKTYWNYCH
int chosenIndex = 0;
const int OPTIONS_NUM = 3;

int GameOverIndex = 0;
const int GO_OPTIONS = 3;
Mix_PlayMusic(menubgm, -1);

while (running)
{
    while (SDL_PollEvent(&e)) {
        if (e.type == SDL_QUIT) { running = false; }
        if (e.type == SDL_KEYDOWN && e.key.keysym.sym == SDLK_ESCAPE) { running = false; } //zamykanie programu
        po wcisnięciu escape
        if (current_state == SAVESCORE)
        {
            if (typing == false)
            {
                SDL_StartTextInput(); //aktywacja pisania
                usernameinput = "";
                typing = true;
            }
            else {
                if (e.type == SDL_TEXTINPUT) //przyjmuje tekst i dodaje do inputa
                {
                    usernameinput += e.text.text;
                }
                else if (e.type == SDL_KEYDOWN)
                {
                    if (e.key.keysym.sym == SDLK_BACKSPACE && usernameinput.length() > 0) //działający klawisz usuwania
                    {
                        usernameinput.pop_back();
                    }
                    else if (e.key.keysym.sym == SDLK_RETURN) //zapis gracza do pliku i przejście do leaderboarda
                    {
                        vector<Player> scores = loadScores();
                        scores.push_back({ usernameinput, score });

                        sort(scores.begin(), scores.end(), [](const Player& a, const Player& b) {
                            return a.HighScore > b.HighScore;
                        });
                    }
                }
            }
        }
    }
}
```

```

        if (scores.size() > 10)
        {
            scores.resize(10);
        }

        saveScores(scores);
        SDL_StopTextInput();
        typing = false;
        current_state = LEADERBOARD;
    }
}
}
}
else if (e.type == SDL_KEYDOWN) {
    if (current_state == MENU)
    {

        if(e.key.keysym.sym == SDLK_UP)
        {
            chosenIndex = (chosenIndex - 1 + OPTIONS_NUM) % OPTIONS_NUM;
        }
        else if (e.key.keysym.sym == SDLK_DOWN)
        {
            chosenIndex = (chosenIndex + 1) % OPTIONS_NUM;
        }
        else if (e.key.keysym.sym == SDLK_RETURN)
        {
            switch(chosenIndex)
            {
                case 0:
                    current_state = PLAYING;
                    Mix_HaltMusic();
                    Mix_PlayMusic(gamebgm, -1);

                    score = 0;
                    snake_size = 3;
                    rq.clear();

                    head = { WINDOW_WIDTH / 2, WINDOW_HEIGHT / 2, 32, 32 };
                    dir = RIGHT;

                    artefakty(artifacts);
                    ScoreHUD.clean();

                    break;
                case 1:
                    current_state = LEADERBOARD;
                    break;
                case 2:
                    running = false;
            }
        }
    }
}
}

```

TOP 10 GRACZY

1. LADYGAGA - 63
2. MIKIMUR - 54
3. GRZECHU - 50
4. JUSTYNKA - 47
5. BRUNO - 43
6. MIKIK - 40
7. MURCZYMIK - 37
8. BOBEREK - 28
9. MIKIOP - 23
10. LUSIA - 17

ABY POWRODZIĆ DO MENU UŻYJ 'M'

WPISZ SWOJĄ NAZWĘ:

ALEKSANDRA

- **Inteligentna generacja artefaktów**

Zamiast statycznych punktów, gra losuje położenie skarbów w obrębie okna. Zastosowaliśmy funkcję, która czyści wektor i dodaje nowy obiekt o określonych wymiarach zapewniając dynamikę rozgrywki, a także zmienia teksturę z każdą teleportacją.

```
void artefakty(vector<SDL_Rect>& artifacts)
{
    artifacts.clear();

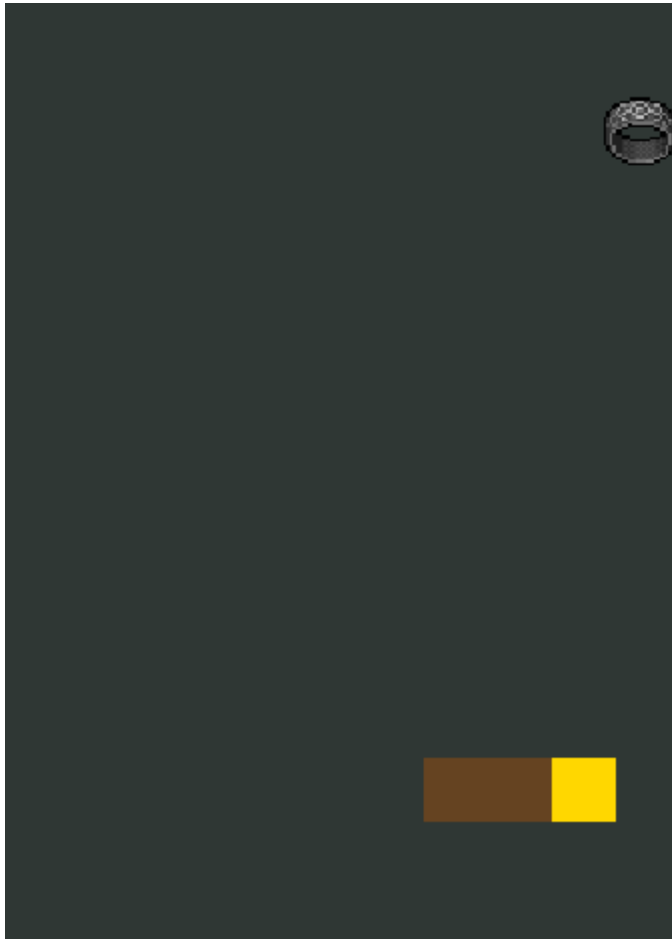
    artifacts.push_back({
```

```

        rand() % (WINDOW_WIDTH - 32), // Losowy X w granicach okna
        rand() % (WINDOW_HEIGHT - 32), // Losowy Y w granicach okna
        48, 48
    });
    textRand = (rand() % 5) + 1;

}

```



- **Wykrywanie kolizji i logika "śmierci"**

Zaimplementowaliśmy dwa rodzaje kolizji: z otoczeniem (ściany) oraz z samym sobą. Wykorzystanie pętli `for_each` pozwala sprawdzić każdy segment ogona względem pozycji głowy.

```

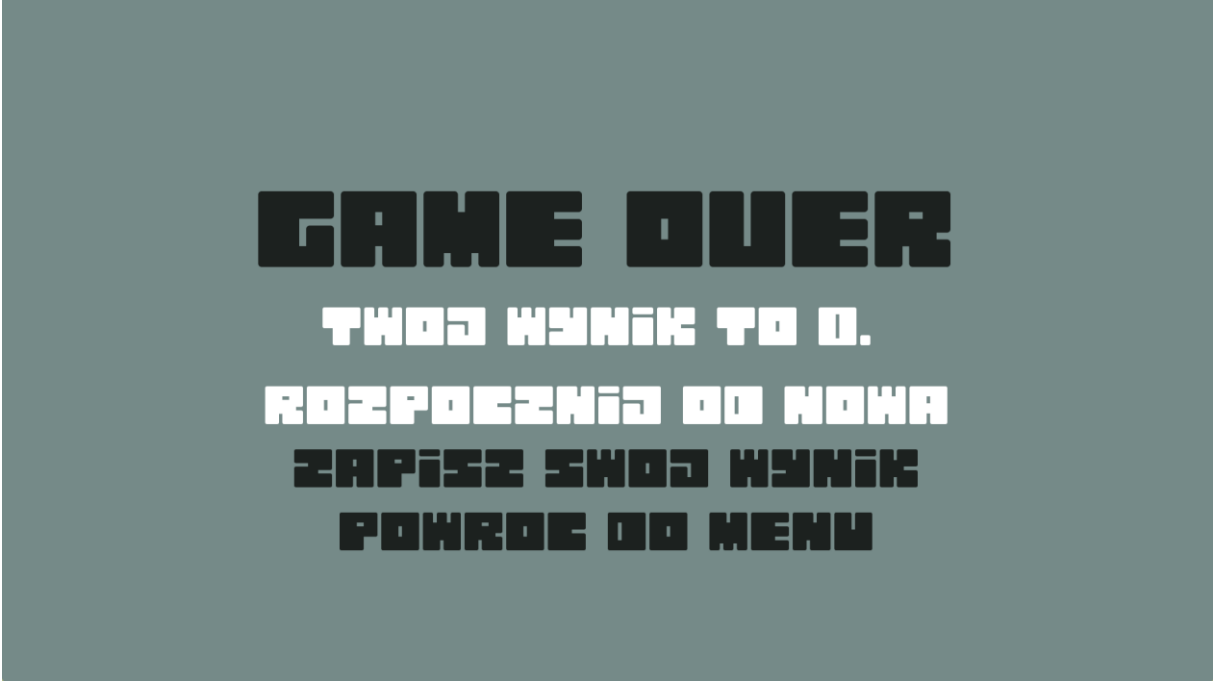
//Sam ze soba - zakoncz gre
for_each(rq.begin(), rq.end(), [&](auto& snake_segment) {
    if (head.x == snake_segment.x && head.y == snake_segment.y)
    {
        Mix_HaltMusic();
        Mix_PlayMusic(menubgm, -1);
        current_state = GAMEOVER;
    }
});

//Ze sciana - zakoncz gre
if (head.x < 0 || head.x > WINDOW_WIDTH - 32 || head.y < 0 || head.y > WINDOW_HEIGHT - 32)

```

```
{  
    Mix_HaltMusic();  
    Mix_PlayMusic(menubgm, -1);  
  
    current_state = GAMEOVER;  
}
```

Zmienia stan na Game Over screen, wstrzymuje muzykę grającą przy rozgrywce i zmienia ją na muzykę z menu.



GAME OVER
TWOJ WYNIK TO 0.
ROZPOCZNIJ OD NOWA
ZAPISZ SWOJ WYNIK
POWRÓĆ DO MENU

Link do repozytorium

GitHub: https://github.com/Isahaoth/PI_snakeproject

3. Jak włączyć grę?

1. **Otwórz projekt:** Uruchom plik .sln w programie Visual Studio.
2. **Zainstaluj bibliotekę SDL2, SDL_image, SDL_ttf oraz SDL_mixer:** Przykładowy tutorial na podstawie jakiego instalacja przebiegała u nas - (Postępować z tutorialiem albo według readme na githubie)

<https://youtu.be/z06-ufmX2XQ?si=JoASHv7BPbPvwmJf>

3. **Sprawdź pliki:** Upewnij się, że w folderze z grą jest folder z obrazkami (assets) oraz pliki z czcionką (blocky.ttf) i muzyką.
4. **Uruchom:** Kliknij zieloną strzałkę (**F5**) w Visual Studio, aby skompilować i włączyć grę.
- 5.

Sterowanie:

- **Strzałki:** Ruch wężem i chodzenie po menu.
- **Enter:** Wybór opcji i zatwierdzanie nicku.
- **M:** Powrót do menu z tabeli wyników.
- **Escape:** Wyjście z gry.

4. Wnioski:

Praca nad projektem pokazała nam, że najważniejsza jest poprawna konfiguracja bibliotek oraz pilnowanie, aby wszystkie pliki, takie jak czcionki czy pliki DLL, znajdowały się w folderze z grą. Nauczyliśmy się pisać kod tak, aby był odporny na błędy braku danych, co zapobiegło nagłemu wyłączaniu się programu. Zrozumieliśmy też, jak ważne jest zwalnianie pamięci po nieużywanych obrazkach i tekstach, żeby gra działała płynnie. Okazało się, że nawet prosta gra staje się dużo fajniejsza, jeśli nada się jej ciekawy motyw graficzny i spójną kolorystykę.